

OTA SWU Signing Algorithm Selection Procedure (RSA-3072 / ML-DSA-87)

1. Overview

Item	Description
Scope	CMS signature of the SWUpdate OTA package (.swu) and the trusted certificate the device verifies it with
Branch	shiba-meta-secure-boot ota-mldsa87-develop (base: develop)
Signed content	sw-description inside the .swu (CMS, signed-attributes digest SHA-384)
Supported algorithms	RSA-3072, ML-DSA-87
On-device verification	2026-10-08, grey board (mode-switch OTA, rejection, SCLI local-install)

Two things are chosen at build time.

- **Certificates the device will trust** (SWU_VERIFY_MODE): installed in the image rootfs as /usr/share/swupdate/swu-sign-cert.pem. Takes effect from the **next OTA** after this image is installed.
- **Algorithm that signs this .swu** (SWU_SIGN_ALG): must be an algorithm trusted by the image **currently installed** on the device.

2. Modes and Outputs

SWU_VERIFY_MODE	SWU_SIGN_ALG	Device trusted certificates	.swu signature	swupdate OpenSSL config
rsa3072 (default)	rsa3072 (fixed)	RSA-3072 only	RSA-3072	/etc/ssl/openssl.cnf (FIPS only, unchanged)
dual	rsa3072 (default)	RSA-3072 + ML-DSA-87	RSA-3072	/etc/ssl/swupdate.cnf
dual	mldsa87	RSA-3072 + ML-DSA-87	ML-DSA-87	/etc/ssl/swupdate.cnf
mldsa87	mldsa87 (fixed)	ML-DSA-87 only	ML-DSA-87	/etc/ssl/swupdate.cnf

- In `rsa3072` or `mldsa87` mode, specifying the other signing algorithm stops the build at parse time.
- In `dual`, changing `SWU_SIGN_ALG` leaves the image content (certificates, config files) unchanged; only the .swu signature changes.
- `dual` and `mldsa87` images also install a swupdate-only OpenSSL config, `/etc/ssl/swupdate.cnf`, and `/etc/swupdate/conf.d/10-openssl`. The FIPS 3.1.2 provider has no ML-DSA, so the default provider is used; in these two modes **swupdate signature verification is outside the FIPS boundary**. All other programs keep using `/etc/ssl/openssl.cnf` (FIPS only).

3. Signing Keys and Certificates

3.1 Location

The build does not generate keys. Place them in advance under `SIGN_KEY_PATH` on the build machine (default `~/STM32AP_KeyGen`). Master copies are kept in `/opt/STM32AP_KeyGen`.

```
${SIGN_KEY_PATH}/
├── swu-sign-3072/          # RSA-3072
│   ├── swu-sign-key.pem    # private key (0600, no passphrase)
│   └── swu-sign-cert.pem   # self-signed certificate
└── swu-sign-mldsa87/      # ML-DSA-87
    ├── swu-sign-key.pem
    └── swu-sign-cert.pem
```

Mode	Required files
rsa3072	swu-sign-3072/ key + certificate
mldsa87	swu-sign-mldsa87/ key + certificate
dual	Both certificates + the key for <code>SWU_SIGN_ALG</code>

- The device trusts the **certificate file itself**. With multiple build machines, copy the same key and certificate to each. If each machine generates its own, devices reject .swu files built on the other machines.
- Store private keys without a passphrase. The build's signing command (`openssl cms -sign -inkey ...`) does not pass one.

3.2 Generating the ML-DSA-87 Key

OpenSSL 3.5 or later is required. The system openssl on orb (Ubuntu 22.04) is 3.0.2 and cannot be used, so use openssl-native (3.5.4) from a tree that has been built once.

```
cd /home/yslim/yocto/tpm-spi/build-openstlinuxweston-shiba-ota-grey
O=$(ls -d tmp-glibc/sysroots-components/*/openssl-native/usr | head -1)
export LD_LIBRARY_PATH=$PWD/$O/lib
OPENSSL=$PWD/$O/bin/openssl
$OPENSSL version                # confirm 3.5 or later

K=~/.STM32AP_KeyGen/swu-sign-mldsa87
mkdir -p $K && cd $K
(umask 077; $OPENSSL genpkey -algorithm ML-DSA-87 -out swu-sign-key.pem)
$OPENSSL req -new -x509 -config /dev/null -key swu-sign-key.pem -out swu-sign-cert.pem \
    -days 7300 -subj "/CN=shiba SWU Key ML-DSA-87"
```

On a Mac, the same commands work with Homebrew OpenSSL (`/opt/homebrew/bin/openssl`, 3.6.x). `/usr/bin/openssl` is LibreSSL and does not support ML-DSA.

3.3 Generating the RSA-3072 Key (only when creating a new one)

Skip this section if you keep using the existing `swu-sign-3072/`. Changing the key means existing devices cannot accept new .swu files.

```
K=~/.STM32AP_KeyGen/swu-sign-3072
mkdir -p $K && cd $K
(umask 077; openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:3072 -out swu-sign-
key.pem)
openssl req -new -x509 -config /dev/null -key swu-sign-key.pem -out swu-sign-cert.pem \
-sha384 -days 7300 -subj "/CN=shiba SWU Key"
```

3.4 Checking

```
cd ~/.STM32AP_KeyGen
$OPENSSL x509 -in swu-sign-3072/swu-sign-cert.pem -noout -subject -enddate -text | grep
-E 'subject|Not After|Public Key Algorithm|Public-Key'
$OPENSSL x509 -in swu-sign-mldsa87/swu-sign-cert.pem -noout -subject -enddate -text | grep
-E 'subject|Not After|Public Key Algorithm'
# key and certificate are a pair (the two hashes must match)
d=swu-sign-mldsa87
$OPENSSL pkey -in $d/swu-sign-key.pem -pubout -outform DER | $OPENSSL dgst -sha256
$OPENSSL x509 -in $d/swu-sign-cert.pem -pubkey -noout | $OPENSSL pkey -pubin -pubout
-outform DER | $OPENSSL dgst -sha256
```

Certificate	Expected
RSA	Public Key Algorithm: rsaEncryption, Public-Key: (3072 bit)
ML-DSA	Public Key Algorithm: ML-DSA-87

The build runs the same checks and stops if they fail.

4. Build

4.1 local.conf Settings

Add the desired combination at the end of `conf/local.conf` in the build directory. With nothing added, the mode is `rsa3072`.

```
# Trust RSA-3072 only, RSA signature (same as default)
SWU_VERIFY_MODE = "rsa3072"

# Trust both, RSA signature
SWU_VERIFY_MODE = "dual"

# Trust both, ML-DSA signature
SWU_VERIFY_MODE = "dual"
SWU_SIGN_ALG = "mldsa87"

# Trust ML-DSA-87 only, ML-DSA signature
SWU_VERIFY_MODE = "mldsa87"
```

To use a `SIGN_KEY_PATH` other than the default (`~/.STM32AP_KeyGen`), add `SIGN_KEY_PATH = "/path"` to the same file.

4.2 Checking the Output Signature

The .swu file name does not include the signing algorithm, and builds with the same version overwrite the same name. Check the signature before deployment, and when archiving, add the algorithm to the name (e.g. ...-r160-20261007-rsa3072.swu).

```
mkdir x && cd x && cpio -idv < ../swu-image-secure-<MACHINE>-<version>.swu
openssl cms -cmsout -print -inform DER -in sw-description.sig | grep -A1 -E
'signatureAlgorithm:|subject:'
```

Signature	signatureAlgorithm	sw-description.sig size
RSA-3072	rsaEncryption	about 2 KB
ML-DSA-87	ML-DSA-87 (2.16.840.1.101.3.4.3.19)	about 12 KB

To also verify against the certificate:

```
openssl cms -verify -inform DER -in sw-description.sig -content sw-description -binary \
-CAfile ~/STM32AP_KeyGen/swu-sign-mldsa87/swu-sign-cert.pem -purpose any -out /dev/null
# CMS Verification successful
```

openssl here must be 3.5 or later (section 3.2).

5. Switching the Signing Algorithm

5.1 Rule

Sign the .swu with an algorithm trusted by the image currently installed on the device. The mode of a newly built image takes effect only after it is installed.

Current device	Accepted signatures	Installable image modes
rsa3072	RSA only	rsa3072, dual (RSA-signed)
dual	RSA, ML-DSA	Any mode (with a signature that mode allows)
mldsa87	ML-DSA only	mldsa87, dual (ML-DSA-signed)

Therefore `rsa3072` and `mldsa87` cannot be switched directly; **always go through `dual`**.

5.2 RSA-3072 → ML-DSA-87

Step	local.conf	.swu signature	Device after install
1	SWU_VERIFY_MODE = "dual"	RSA (default)	dual (trusts RSA + ML-DSA)
2	SWU_VERIFY_MODE = "mldsa87"	ML-DSA (automatic)	mldsa87 (trusts ML-DSA only)

Deploy the step 2 .swu **only after the step 1 OTA has completed on all target devices**. Devices that missed step 1 reject the step 2 .swu.

5.3 ML-DSA-87 → RSA-3072 (rollback)

Step	local.conf	.swu signature	Device after install
1	SWU_VERIFY_MODE = "dual", SWU_SIGN_ALG = "mldsa87"	ML-DSA	dual
2	SWU_VERIFY_MODE = "rsa3072"	RSA (automatic)	rsa3072 (back to the original FIPS config)

5.4 Changing Only the Signature in dual

A `dual` device accepts a `.swu` built with only `SWU_SIGN_ALG` changed. Nothing changes on the device.

5.5 Installing by Flashing

A factory install that writes the image directly with STM32CubeProgrammer has no previous image, so there is no ordering constraint. Flash an image built in the desired mode. Subsequent OTAs follow the rule in section 5.1.

5.6 When Sent in the Wrong Order

The device does not install and keeps the current image. `swupdate` log:

```
START Software Update started !
FAILURE ERROR : Signature verification failed
FAILURE ERROR : Compatible SW not found
FATAL_FAILURE Image invalid or corrupted. Not installing ...
```

`ustate=0`, and the `swupdate` service keeps running. Re-sign with an algorithm the device trusts and send it again (in `dual`, change only `SWU_SIGN_ALG`; in a single mode, rebuild for that mode).

6. Device Check

Run as root on the device.

```
# All trusted certificates (the x509 command shows only the first). rsa3072 devices have no
swupdate.cnf
C=/etc/ssl/swupdate.cnf; [ -f $C ] || C=/etc/ssl/openssl.cnf
OPENSSL_CONF=$C openssl storeutl -noout -certs -text /usr/share/swupdate/swu-sign-cert.pem \
| grep -E '^[0-9]+: |Subject:|Public Key Algorithm|Total found'

# Both must exist for dual / mldsa87 (absent for rsa3072)
ls -l /etc/ssl/swupdate.cnf /etc/swupdate/conf.d/10-openssl

# Did the swupdate daemon get swupdate.cnf
for P in $(pidof swupdate); do tr '\0' '\n' < /proc/$P/environ | grep OPENSSL_CONF; done
```

On a `dual` or `mldsa87` device, printing the certificates without `OPENSSL_CONF=/etc/ssl/swupdate.cnf` gives `X509_PUBKEY_get0:decode error` on the ML-DSA certificate. This is expected: the default config (FIPS only) has no ML-DSA.

7. Notes

- **Scope:** Only the OTA package signature changes. The boot chain (TF-A/FIP, FIT image signature RSA-2048, ROM ECDSA) and the payload hashes inside the .swu (SHA-256) are unchanged.
- **FIPS boundary:** In `dua1` and `m1dsa87`, swupdate cryptographic operations (including RSA signature verification) may use the default provider and are outside the FIPS boundary. Only `rsa3072` keeps the original FIPS config.